2026-05-06-p2-f22-aider-git-auto-commit

P2-F22 — Aider Git Auto-Commit Per Change Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (`- [ ]`) syntax for tracking.

> **Programme position:** F22 is the **second** Phase 2 feature of CLI-Agent Fusion. Task T01 advances PROGRESS.md from "Phase 2: F21 closed; F22 next candidate" to "Phase 2 of CLI-Agent Fusion programme: F22 (Aider Git Auto-Commit) in flight" and adds the F22 evidence header to `docs/improvements/07_phase_2_evidence.md` (created in F21).

Goal: Ship a real, end-to-end **per-edit git auto-commit** facility for the HelixCode CLI agent, modelled on aider. F22 adds an `internal/autocommit/` package with `AutoCommitter` (atomic-bool enabled flag + `llm.Provider`-backed `MessageSummariser` + deterministic fallback + `Git` thin wrapper over `os/exec` + `SecretFilter` regex pre-pass) + `CommitContext` value type + sentinel errors + `EnvVarName` / `CoAuthorTrailer` / `SkipParamKey` constants. Extends `tools/registry.go::Execute` with a NEW post-Execute hook (`fireAutoCommit`) adjacent to the existing F13 LSP-auto-trigger; the hook fires ONLY when `execErr == nil` AND F21 `RequiresApproval()` level ∈ `{LevelEdit, LevelAll}` AND `params["_helix_skip_git_commit"] != true`. Adds a `/git_auto_commit` slash command (`status` / `on` / `off` / `show`); NO cobra subcommand. Runtime opt-out via `/git_auto_commit off` is structurally enforced via `atomic.Bool` swap; the next `MaybeCommit` call sees the new state. Co-author trailer (`Co-Authored-By: HelixCode <noreply@helixcode.dev>`) appended to EVERY auto-commit unconditionally.

Architecture: New `internal/autocommit/` package with `types.go` (`CommitContext` + `CommitResult` + `Options` + sentinel errors `ErrNotGitRepo`/`ErrCommitFailed`/`ErrLLMUnavailable` + `EnvVarName`/`CoAuthorTrailer`/`SkipParamKey` constants), `git.go` (`Git` thin wrapper over `os/exec`: `IsRepo` / `StatusPorcelain` / `DiffStaged` / `DiffUnstaged` / `Add` / `Commit` / `HeadSHA`), `summariser.go` (`MessageSummariser` interface + `LLMSummariser` impl + `DeterministicFallback` + `Summarise(ctx, diff, toolName, paths) string` chain), `secret_filter.go` (regex strip of AKIA / sk- / xoxb / gh[pousr]_ patterns per CONST-042), `committer.go` (`AutoCommitter` with `atomic.Bool` enabled + `MaybeCommit(ctx, cctx) (CommitResult, error)` + `SetEnabled(bool)` + `Enabled() bool` + `IsGitRepo() bool`). New `internal/commands/git_auto_commit_command.go` for the `/git_auto_commit` slash command. Two existing files get small additions: `internal/tools/registry.go` (1) `ToolRegistry` + `autoCommitter` field, (2) `SetAutoCommitter(c)` setter, (3) `Execute` gains a NEW post-success call to `fireAutoCommit(ctx, name, params, tool, result)` adjacent to `triggerLSPAfterEdit`; `cmd/cli/main.go` (1) read `os.Getenv("HELIXCODE_GIT_AUTO_COMMIT")` for initial enabled state, (2) construct `autocommit.AutoCommitter` adjacent to F21 wiring, (3) `c.toolRegistry.SetAutoCommitter(c.autoCommitter)` + register `/git_auto_commit` slash. Plus the per-tool mutated-paths derivation table in §3.5 of the spec applied inside `fireAutoCommit`.

Tech Stack: Go 1.26, testify v1.11, zap (already direct), `internal/llm.Provider` (in-tree). **Zero new external deps** (`os/exec`, `regexp`, `sync/atomic`, `context`, `errors`, `fmt`, `strings`, `time` are stdlib). `go mod tidy` after T08 must produce no diff in either `go.mod` or `go.sum`. T09's verification step asserts this loudly.

Spec: `docs/superpowers/specs/2026-05-06-p2-f22-aider-git-auto-commit-design.md` (commit `8be7fba`).

Working directory for `go` commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F22 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/autocommit internal/commands/
        git_auto_commit_command.go && echo BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — F22 can degenerate in five ways: (a) `git commit` returns success but `git status --porcelain` for the supposedly-staged paths is non-empty (the committer staged the wrong paths or the commit silently dropped files); (b) the commit message is a static template that doesn't reflect the actual diff (the summariser concatenates `toolName + "edit"` without ever calling the LLM, OR the LLM is called but its response is discarded); (c) auto-commit fires on tools that didn't actually mutate any tracked file (the `RequiresApproval()` filter is incomplete OR the porcelain check is skipped, so the committer attempts to commit on a clean tree); (d) env var `HELIXCODE_GIT_AUTO_COMMIT=off` is honoured at startup but `/git_auto_commit on` runtime change isn't reflected in the next edit (the slash mutates a struct field that `MaybeCommit` doesn't re-read on entry); (e) commit messages leak secrets from the diff (CONST-042). The five "what counts as auto-commit works" criteria — (1) real `WriteFileTool` invoked through real registry under default-on auto-commit produces a real commit (PHASE-A: `git log -1 --format=%H` changes; subject non-empty + ≤72 chars; co-author trailer present in `--format=%B`; `git status --porcelain` empty for the path; `git show --stat HEAD` lists the path); (2) the commit subject equals the LLM-fake-provider's sentinel response (PHASE-B: proves LLM round-trip happened); (3) `read_file` (LevelReadOnly) does NOT trigger a commit (PHASE-C: `git log -1 --format=%H` unchanged); (4) `HELIXCODE_GIT_AUTO_COMMIT=off` produces no commit AND `git status --porcelain` shows the file dirty (PHASE-D); (5) `committer.SetEnabled(true)` after starting off causes the very next call to commit (PHASE-E: SHA before/after differential observable) — are each tested with both unit assertions AND a Challenge phase. PHASE-F additionally asserts per-edit `_helix_skip_git_commit:true` is honoured (no commit + dirty tree). Optional PHASE-G feeds a synthetic diff with a fake `sk-AAAA...` key and asserts the message contains `[REDACTED]`. The Challenge harness uses positive evidence: SHA equality (`git log -1 --format=%H` before/after differential), filesystem-state assertions (`git status --porcelain` empty/non-empty), trailer-presence assertion (`git log -1 --format=%B` contains `Co-Authored-By: HelixCode <noreply@helixcode.dev>`), `git show --stat HEAD` lists exactly the expected paths, LLM-fake-provider sentinel equality (PHASE-B), runtime-toggle observable in next-call SHA (PHASE-E). Byte-evidence mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.

Why this is consequential: auto-commit is the user-visible artefact of every edit the agent makes. F21 is the safety surface (deny bad edits); F22 is the legibility surface (record good edits).

Without F22, the user has to manually `git diff` after each session to see what the agent did, and manually `git commit` to lock in the work; with F22, the history materialises automatically and is reviewable, revertable, and `git bisect`-able. F22's discriminating tests are: (i) PHASE-A's `git status --porcelain` empty AND `git show --stat HEAD` lists the path (proves real commit, not metadata-only); (ii) PHASE-B's subject == fake-LLM sentinel (proves LLM-call-then-use, not hardcoded fallback); (iii) PHASE-C's SHA-unchanged after read_file (proves the level filter is correct); (iv) PHASE-D's dirty tree after env-off (proves env opt-out reached MaybeCommit, not just slash status); (v) PHASE-E's SHA differential before/after SetEnabled(true) (proves runtime change is read on entry, not at construct time). All five must produce positive evidence; none can be satisfied by absence-of-error.

Task list

- [ ] P2-F22-T01 — bootstrap F22 evidence section + advance PROGRESS to F22
- [ ] P2-F22-T02 — `internal/autocommit/types.go`: CommitContext + CommitResult + Options + sentinels + EnvVarName/CoAuthorTrailer/SkipParamKey constants (TDD)
- [ ] P2-F22-T03 — `internal/autocommit/git.go`: thin git wrapper (IsRepo/StatusPorcelain/DiffStaged/DiffUnstaged/Add/Commit/HeadSHA) (TDD with real tempdir + git init)
- [ ] P2-F22-T04 — `internal/autocommit/summariser.go` + `secret_filter.go`: LLM-driven summariser + deterministic fallback + secret-pattern strip (TDD)
- [ ] P2-F22-T05 — `internal/autocommit/committer.go`: AutoCommitter.MaybeCommit + atomic.Bool enabled state (TDD)
- [ ] P2-F22-T06 — registry.go: SetAutoCommitter + post-Execute fireAutoCommit hook + per-tool mutated-paths derivation (TDD; coverage table-test)
- [ ] P2-F22-T07 — `/git_auto_commit` slash command (status/on/off/show) (TDD)
- [ ] P2-F22-T08 — main.go wiring + integration test
- [ ] P2-F22-T09 — Challenge harness (5+1 phases: default-on + LLM-summary-accurate + non-edit-no-op + env-off + runtime-toggle + per-edit-skip [+ optional PHASE-G secret filter]) + close-out + push 4 remotes non-force

---

Task 1: Bootstrap F22 evidence

Append F22 section header to `docs/improvements/07_phase_2_evidence.md` (already created in F21-T01) with spec SHA `8be7fba`. Update PROGRESS.md current focus from "Phase 2 of CLI-Agent Fusion programme: F21 closed; F22 next candidate" to "Phase 2 of

CLI-Agent Fusion programme: F22 (Aider Git Auto-Commit Per Change) in flight". Insert F22 task list (9 items). Verify zero new external deps:

```
cd HelixCode && grep -E "autocommit|HELIXCODE_GIT_AUTO_COMMIT"
        go.mod && echo "UNEXPECTED" || echo "clean"
```

Update CONTINUATION.md root-level mid-flight section with F22 in-flight status (will be updated per task by T02-T09 commits).

Commit: `docs(P2-F22-T01): bootstrap F22 evidence + advance PROGRESS to F22 (Aider Git Auto-Commit).`

Task 2: types.go (TDD)

Files: new `helix_code/internal/autocommit/types.go`, new `helix_code/internal/autocommit/types_test.go`.

Define: - `CommitContext struct { ToolName string; Args map[string]interface{}; MutatedPaths []string; SkipRequested bool }`.- `CommitResult struct { SHA string; Subject string; Files []string; Skipped bool; Reason string }`.- `Options struct { Enabled bool; Provider llm.Provider; WorkingDir string; Logger *zap.Logger; NowFunc func() time.Time }`.- Constants: `EnvVarName = "HELIXCODE_GIT_AUTO_COMMIT"`, `CoAuthorTrailer = "Co-Authored-By: HelixCode <noreply@helixcode.dev>"`, `SkipParamKey = "_helix_skip_git_commit"`. - Sentinel errors: `ErrNotGitRepo`, `ErrCommitFailed`, `ErrLLMUnavailable`.

Failing tests FIRST:

```
func TestEnvVarName_Pin(t *testing.T) {
    require.Equal(t, "HELIXCODE_GIT_AUTO_COMMIT", EnvVarName)
}

func TestCoAuthorTrailer_Pin(t *testing.T) {
    require.Equal(t, "Co-Authored-By: HelixCode
        <noreply@helixcode.dev>", CoAuthorTrailer)
}

func TestSkipParamKey_Pin(t *testing.T) {
    require.Equal(t, "_helix_skip_git_commit", SkipParamKey)
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{ErrNotGitRepo, ErrCommitFailed,
        ErrLLMUnavailable} {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}
```

Subject: `feat(P2-F22-T02): autocommit types - CommitContext + CommitResult + Options + sentinels + constants (TDD).`

Task 3: git.go thin wrapper (TDD with real tempdir)

Files: new `helix_code/internal/autocommit/git.go`, new `helix_code/internal/autocommit/git_test.go`.

`git.go`:

```
type Git struct {
    workingDir string
    log        *zap.Logger
}

func NewGit(workingDir string, log *zap.Logger) *Git

func (g *Git) IsRepo(ctx context.Context) (bool, error)
func (g *Git) StatusPorcelain(ctx context.Context) (string,
    error)
func (g *Git) DiffStaged(ctx context.Context) (string, error)
func (g *Git) DiffUnstaged(ctx context.Context) (string, error)
func (g *Git) Add(ctx context.Context, paths ...string) error
func (g *Git) Commit(ctx context.Context, message string) (sha
    string, err error)
func (g *Git) HeadSHA(ctx context.Context) (string, error)

// run is the single shell-out helper; methods are one-line
    wrappers.
func (g *Git) run(ctx context.Context, args ...string) (string,
    error) {
    cmd := exec.CommandContext(ctx, "git", args...)
    cmd.Dir = g.workingDir
    out, err := cmd.CombinedOutput()
    if err != nil {
        return "", fmt.Errorf("git %s: %w (output: %s)",
        strings.Join(args, " "), err, string(out))
    }
    return string(out), nil
}
```

Failing tests FIRST (real tempdir, real git operations):

```
func setupRealGitRepo(t *testing.T) string {
    t.Helper()
    dir := t.TempDir()
    cmd := exec.Command("git", "init")
    cmd.Dir = dir
```

```
    require.NoError(t, cmd.Run())
    cmd = exec.Command("git", "config", "user.email",
        "test@helixcode.dev")
    cmd.Dir = dir
    require.NoError(t, cmd.Run())
    cmd = exec.Command("git", "config", "user.name", "Test")
    cmd.Dir = dir
    require.NoError(t, cmd.Run())
    return dir
}

func TestGit_IsRepo_True_InsideRepo(t *testing.T) {
    dir := setupRealGitRepo(t)
    g := NewGit(dir, zap.NewNop())
    ok, err := g.IsRepo(context.Background())
    require.NoError(t, err)
    require.True(t, ok)
}

func TestGit_IsRepo_False_OutsideRepo(t *testing.T) {
    g := NewGit(t.TempDir(), zap.NewNop())
    ok, _ := g.IsRepo(context.Background())
    require.False(t, ok)
}

func TestGit_StatusPorcelain_DirtyAfterWrite(t *testing.T) {
    dir := setupRealGitRepo(t)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
    g := NewGit(dir, zap.NewNop())
    out, err := g.StatusPorcelain(context.Background())
    require.NoError(t, err)
    require.Contains(t, out, "x.txt")
}

func TestGit_AddCommitHeadSHA_RoundTrip(t *testing.T) {
    dir := setupRealGitRepo(t)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
    g := NewGit(dir, zap.NewNop())
    require.NoError(t, g.Add(context.Background(), "x.txt"))
    sha, err := g.Commit(context.Background(), "test
        commit\n\nbody")
    require.NoError(t, err)
    require.NotEmpty(t, sha)
    head, err := g.HeadSHA(context.Background())
    require.NoError(t, err)
    require.Equal(t, sha, head)
```

```
    // post-condition: working tree clean
    out, _ := g.StatusPorcelain(context.Background())
    require.Empty(t, strings.TrimSpace(out))
}

func TestGit_DiffStaged_NonEmptyAfterAdd(t *testing.T) {
    dir := setupRealGitRepo(t)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello\n"), 0644))
    g := NewGit(dir, zap.NewNop())
    require.NoError(t, g.Add(context.Background(), "x.txt"))
    diff, err := g.DiffStaged(context.Background())
    require.NoError(t, err)
    require.Contains(t, diff, "+hello")
}
```

Non-obvious call: tests use real `git init` + real `exec.Command("git", "init")` — NO mocks of git. CONST-035 demands real git operations in integration-flavour tests; the unit tests for the wrapper are integration-flavour by necessity (mocking git defeats the wrapper's purpose).

Subject: `feat(P2-F22-T03): autocommit git wrapper - IsRepo/StatusPorcelain/DiffStaged/Add/Commit/HeadSHA (real-git TDD).`

Task 4: summariser.go + secret_filter.go (TDD)

Files: new `helix_code/internal/autocommit/summariser.go`, new `helix_code/internal/autocommit/summariser_test.go`, new `helix_code/internal/autocommit/secret_filter.go`, new `helix_code/internal/autocommit/secret_filter_test.go`.

`summariser.go`:

```
type MessageSummariser interface {
    Summarise(ctx context.Context, diff, toolName string, paths
        []string) string
}

type LLMSummariser struct {
    provider llm.Provider
    timeout  time.Duration
}

func NewSummariser(p llm.Provider) MessageSummariser {
    if p == nil {
        return &DeterministicFallback{}
    }
    return &LLMSummariser{provider: p, timeout: 5 * time.Second}
}
```

```
const summarisePrompt = "Summarise this diff in 50-72 chars
        (imperative voice, no period):\n\n"

const maxDiffBytes = 8 * 1024
const maxSubjectChars = 72

func (s *LLMSummariser) Summarise(ctx context.Context, diff,
        toolName string, paths []string) string {
    if len(diff) > maxDiffBytes {
        diff = diff[:maxDiffBytes]
    }
    cctx, cancel := context.WithTimeout(ctx, s.timeout)
    defer cancel()
    req := &llm.LLMRequest{
        Model:    s.provider.GetModels()[0].ID,
        Messages: []llm.Message{{Role: "user", Content:
        summarisePrompt + diff}},
    }
    resp, err := s.provider.Generate(cctx, req)
    if err != nil || resp == nil {
        return (&DeterministicFallback{}).Summarise(ctx, diff,
        toolName, paths)
    }
    out := strings.TrimSpace(resp.Content)
    if out == "" {
        return (&DeterministicFallback{}).Summarise(ctx, diff,
        toolName, paths)
    }
    if len(out) > maxSubjectChars {
        out = out[:maxSubjectChars]
    }
    return out
}

type DeterministicFallback struct{}

func (DeterministicFallback) Summarise(_ context.Context, _,
        toolName string, paths []string) string {
    msg := fmt.Sprintf("Auto-edit: %s on %s", toolName,
        strings.Join(paths, ", "))
    if len(msg) > maxSubjectChars {
        msg = msg[:maxSubjectChars]
    }
    return msg
}
```

secret_filter.go:

```
type SecretFilter struct {
    patterns []*regexp.Regexp
}

func NewSecretFilter() *SecretFilter {
    return &SecretFilter{
        patterns: []*regexp.Regexp{
            regexp.MustCompile(`AKIA[0-9A-Z]{16}`),
            regexp.MustCompile(`sk-[A-Za-z0-9]{20,}`),
            regexp.MustCompile(`xox[baprs]-[A-Za-z0-9-]{10,}`),
            regexp.MustCompile(`gh[pousr]_[A-Za-z0-9]{36}`),
        },
    }
}

func (f *SecretFilter) Filter(s string) string {
    for _, p := range f.patterns {
        s = p.ReplaceAllString(s, "[REDACTED]")
    }
    return s
}
```

Failing tests FIRST:

```
type fakeProvider struct {
    response string
    err      error
    calls    int
}
func (f *fakeProvider) GetType() llm.ProviderType { return
        "fake" }
func (f *fakeProvider) GetName() string           { return
        "fake" }
func (f *fakeProvider) GetModels() []llm.ModelInfo {
    return []llm.ModelInfo{{ID: "fake-model"}}
}
func (f *fakeProvider) GetCapabilities() []llm.ModelCapability {
        return nil }
func (f *fakeProvider) Generate(ctx context.Context, req
        *llm.LLMRequest) (*llm.LLMResponse, error) {
    f.calls++
    if f.err != nil { return nil, f.err }
    return &llm.LLMResponse{Content: f.response}, nil
}
// ... GenerateStream/IsAvailable/etc nil-impl ...

func TestSummariser_LLMSuccess_UsesProviderResponse(t
        *testing.T) {
    p := &fakeProvider{response: "FAKE_LLM_RESPONSE_42"}
```

```
    s := NewSummariser(p)
    got := s.Summarise(context.Background(), "diff body",
        "fs_edit", []string{"x.go"})
    require.Equal(t, "FAKE_LLM_RESPONSE_42", got)
    require.Equal(t, 1, p.calls)
}

func TestSummariser_LLMError_FallsBackToDeterministic(t
        *testing.T) {
    p := &fakeProvider{err: errors.New("boom")}
    s := NewSummariser(p)
    got := s.Summarise(context.Background(), "diff body",
        "fs_edit", []string{"x.go"})
    require.Equal(t, "Auto-edit: fs_edit on x.go", got)
}

func TestSummariser_LLMEmpty_FallsBackToDeterministic(t
        *testing.T) {
    p := &fakeProvider{response: "   \n\t"}
    s := NewSummariser(p)
    got := s.Summarise(context.Background(), "diff body",
        "fs_edit", []string{"x.go"})
    require.Equal(t, "Auto-edit: fs_edit on x.go", got)
}

func TestSummariser_LLMTooLong_TruncatedAt72(t *testing.T) {
    long := strings.Repeat("A", 100)
    p := &fakeProvider{response: long}
    s := NewSummariser(p)
    got := s.Summarise(context.Background(), "diff", "t",
        []string{"f"})
    require.Equal(t, 72, len(got))
}

func TestDeterministicFallback_Format_ByteForByte(t *testing.T) {
    var d DeterministicFallback
    require.Equal(t, "Auto-edit: fs_edit on foo.go, bar.go",
        d.Summarise(context.Background(), "", "fs_edit",
        []string{"foo.go", "bar.go"}))
}

func TestSecretFilter_AKIA_Redacted(t *testing.T) {
    f := NewSecretFilter()
    out := f.Filter("key=AKIAABCDEFGHIJKLMNOP rest")
    require.Equal(t, "key=[REDACTED] rest", out)
}

func TestSecretFilter_OpenAI_Redacted(t *testing.T) {
```

```
    f := NewSecretFilter()
    out := f.Filter("key=sk-abcdefghij1234567890abcdef rest")
    require.Contains(t, out, "[REDACTED]")
    require.NotContains(t, out, "sk-abc")
}

func TestSecretFilter_Slack_Redacted(t *testing.T) {
    f := NewSecretFilter()
    out := f.Filter("xoxb-1234567890ab")
    require.Contains(t, out, "[REDACTED]")
}

func TestSecretFilter_GitHub_Redacted(t *testing.T) {
    f := NewSecretFilter()
    out := f.Filter("ghp_" + strings.Repeat("a", 36))
    require.Contains(t, out, "[REDACTED]")
}
```

Non-obvious call: the fake provider is an in-package test stub, NOT in `internal/mocks/`. Per CLAUDE.md, mocks are allowed in unit tests; the in-package stub is the cleaner pattern (no cross-package import; minimal surface).

Subject: `feat(P2-F22-T04): autocommit summariser + secret filter - LLM + deterministic fallback + 4 secret patterns (TDD).`

Task 5: committer.go (TDD with real git tempdir)

Files: new `helix_code/internal/autocommit/committer.go`, new `helix_code/internal/autocommit/committer_test.go`.

`committer.go` per spec §3.3 (full pipeline). Key points: - `atomic.Bool` enabled flag. - `MaybeCommit(ctx, cctx)` is the single entry point with the 11-step pipeline per spec §4. - NEVER returns commit errors as fatal — all errors are wrapped in `ErrCommitFailed` and the caller is expected to log + continue. - Post-condition assertion: `git.HeadSHA()` equals the returned SHA AND `git.StatusPorcelain` for the staged paths is empty.

Failing tests FIRST (full pipeline against real tempdir + real git):

```
func newRealCommitter(t *testing.T, dir string, p llm.Provider,
        enabled bool) *AutoCommitter {
    return NewAutoCommitter(Options{
        Enabled: enabled, Provider: p, WorkingDir: dir, Logger:
        zap.NewNop(),
    })
}

func TestCommitter_DefaultOn_EditCommitsRealCommit(t *testing.T)
        {
    dir := setupRealGitRepo(t)
```

```
    initialCommit(t, dir)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
    c := newRealCommitter(t, dir, &fakeProvider{response:
        "SUMMARY"}, true)
    res, err := c.MaybeCommit(context.Background(),
        CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
    })
    require.NoError(t, err)
    require.False(t, res.Skipped)
    require.NotEmpty(t, res.SHA)
    require.Equal(t, "SUMMARY", res.Subject)

    // Real evidence: git log shows the commit, working tree
        clean.
    out, _ := exec.Command("git", "-C", dir, "log", "-1", "--
        format=%H").Output()
    require.Equal(t, res.SHA, strings.TrimSpace(string(out)))
    out, _ = exec.Command("git", "-C", dir, "log", "-1", "--
        format=%B").Output()
    require.Contains(t, string(out), CoAuthorTrailer)
    out, _ = exec.Command("git", "-C", dir, "status", "--
        porcelain").Output()
    require.Empty(t, strings.TrimSpace(string(out)))
}

func TestCommitter_Disabled_SkipsCommit(t *testing.T) {
    dir := setupRealGitRepo(t)
    initialCommit(t, dir)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
    c := newRealCommitter(t, dir, &fakeProvider{response: "S"},
        false)
    res, err := c.MaybeCommit(context.Background(),
        CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
    })
    require.NoError(t, err)
    require.True(t, res.Skipped)
}

func TestCommitter_SkipRequested_Honoured(t *testing.T) {
    dir := setupRealGitRepo(t)
    initialCommit(t, dir)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
```

```
    c := newRealCommitter(t, dir, &fakeProvider{response: "S"},
        true)
    res, _ := c.MaybeCommit(context.Background(), CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
        SkipRequested: true,
    })
    require.True(t, res.Skipped)
    require.Contains(t, res.Reason, "per-edit skip")
}

func TestCommitter_NotAGitRepo_Skips(t *testing.T) {
    dir := t.TempDir()  // NOT a git repo
    c := newRealCommitter(t, dir, nil, true)
    res, _ := c.MaybeCommit(context.Background(), CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
    })
    require.True(t, res.Skipped)
    require.Contains(t, res.Reason, "not a git repo")
}

func TestCommitter_CleanTree_NoChanges(t *testing.T) {
    dir := setupRealGitRepo(t)
    initialCommit(t, dir)
    c := newRealCommitter(t, dir, nil, true)
    res, _ := c.MaybeCommit(context.Background(), CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
    })
    require.True(t, res.Skipped)
    require.Contains(t, res.Reason, "no changes")
}

func TestCommitter_LLMUnavailable_FallsBack_StillCommits(t
        *testing.T) {
    dir := setupRealGitRepo(t)
    initialCommit(t, dir)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
    c := newRealCommitter(t, dir, &fakeProvider{err:
        errors.New("boom")}, true)
    res, err := c.MaybeCommit(context.Background(),
        CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
    })
    require.NoError(t, err)
    require.False(t, res.Skipped)
    require.Contains(t, res.Subject, "Auto-edit:")
}
```

```
func TestCommitter_SetEnabled_AtomicSwap_NextCallSeesNewState(t
        *testing.T) {
    dir := setupRealGitRepo(t)
    initialCommit(t, dir)
    c := newRealCommitter(t, dir, &fakeProvider{response: "S"},
        false)
    require.False(t, c.Enabled())
    c.SetEnabled(true)
    require.True(t, c.Enabled())

    require.NoError(t, os.WriteFile(filepath.Join(dir, "y.txt"),
        []byte("hi"), 0644))
    res, err := c.MaybeCommit(context.Background(),
        CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"y.txt"},
    })
    require.NoError(t, err)
    require.False(t, res.Skipped)
}

func TestCommitter_CoAuthorTrailer_AppendedAlways(t *testing.T) {
    dir := setupRealGitRepo(t)
    initialCommit(t, dir)
    require.NoError(t, os.WriteFile(filepath.Join(dir, "x.txt"),
        []byte("hello"), 0644))
    c := newRealCommitter(t, dir, &fakeProvider{response:
        "subject"}, true)
    _, err := c.MaybeCommit(context.Background(), CommitContext{
        ToolName: "fs_write", MutatedPaths: []string{"x.txt"},
    })
    require.NoError(t, err)
    out, _ := exec.Command("git", "-C", dir, "log", "-1", "--
        format=%B").Output()
    require.Contains(t, string(out), "Co-Authored-By: HelixCode
        <noreply@helixcode.dev>")
}
```

Subject: `feat(P2-F22-T05): autocommit committer - MaybeCommit pipeline + atomic enabled + co-author trailer (real-git TDD).`

Task 6: registry.go SetAutoCommitter + post-Execute hook + mutated-paths derivation (TDD)

Files: modify `helix_code/internal/tools/registry.go`; new `helix_code/internal/tools/registry_autocommit_test.go`.

Steps:

1. Add `autoCommitter *autocommit.AutoCommitter` field to `ToolRegistry`.
2. Add `SetAutoCommitter(c *autocommit.AutoCommitter)` setter (mu-locked, mirrors `SetApprovalManager`).
3. Add `fireAutoCommit(ctx, name, params, tool, result)` helper. Pseudocode:

```
func (r *ToolRegistry) fireAutoCommit(ctx context.Context, name
        string,
    params map[string]interface{}, tool Tool, result
        interface{}) {
    r.mu.RLock()
    c := r.autoCommitter
    r.mu.RUnlock()
    if c == nil {
        return
    }
    level := tool.RequiresApproval()
    if level != approval.LevelEdit && level != approval.LevelAll
        {
        return
    }
    if skip, ok := params[autocommit.SkipParamKey].(bool); ok &&
        skip {
        return
    }
    paths := derivePaths(name, params)
    cctx := autocommit.CommitContext{
        ToolName:     name,
        Args:         params,
        MutatedPaths: paths,
    }
    if _, err := c.MaybeCommit(ctx, cctx); err != nil {
        // Best-effort: log + continue. F22-spec §11 #9.
        // Use the registry's logger if available; otherwise
        zap.NewNop.
    }
}

func derivePaths(toolName string, params map[string]interface{})
        []string {
    switch toolName {
    case "fs_write", "fs_edit", "smart_edit", "notebook_edit":
        if p, ok := params["path"].(string); ok && p != "" {
            return []string{p}
        }
    case "multiedit_commit":
        if edits, ok := params["edits"].([]interface{}); ok {
```

```
            var out []string
            seen := map[string]struct{}{}
            for _, e := range edits {
                if m, ok := e.(map[string]interface{}); ok {
                    if p, ok := m["path"].(string); ok && p !=
        "" {
                        if _, dup := seen[p]; !dup {
                            seen[p] = struct{}{}
                            out = append(out, p)
                        }
                    }
                }
            }
            return out
        }
    case "mapping_edit":
        if p, ok := params["target_file"].(string); ok && p !=
        "" {
            return []string{p}
        }
    }
    return nil // generic fallthrough; porcelain-based discovery
        in committer
}
```

1. Modify `Execute` to call `r.fireAutoCommit(ctx, name, params, tool, result)` adjacent to the existing `triggerLSPAfterEdit` call (AFTER it, BEFORE the function returns). Run only when `execErr == nil`.

Failing tests FIRST:

```
func TestRegistry_FireAutoCommit_NilCommitter_NoOp(t *testing.T)
        {
    reg := NewToolRegistry()
    // No SetAutoCommitter call.
    require.NotPanics(t, func() {
        reg.fireAutoCommit(context.Background(), "fs_write",
            map[string]interface{}{"path": "x.txt"},
        &fakeEditTool{}, nil)
    })
}

func TestRegistry_FireAutoCommit_SkipParamHonoured(t *testing.T)
        {
    spy := &spyCommitter{}
    reg := NewToolRegistry()
    reg.SetAutoCommitter(spy.asReal())
    reg.fireAutoCommit(context.Background(), "fs_write",
```

```
        map[string]interface{}{"path": "x.txt",
        "_helix_skip_git_commit": true},
        &fakeEditTool{}, nil)
    require.Equal(t, 0, spy.calls)
}

func TestRegistry_FireAutoCommit_NonEditLevel_Excluded(t
        *testing.T) {
    spy := &spyCommitter{}
    reg := NewToolRegistry()
    reg.SetAutoCommitter(spy.asReal())
    reg.fireAutoCommit(context.Background(), "read_file",
        map[string]interface{}{"path": "x.txt"},
        &fakeReadOnlyTool{}, nil)
    require.Equal(t, 0, spy.calls)
}

func TestRegistry_FireAutoCommit_EditLevel_Calls(t *testing.T) {
    spy := &spyCommitter{}
    reg := NewToolRegistry()
    reg.SetAutoCommitter(spy.asReal())
    reg.fireAutoCommit(context.Background(), "fs_write",
        map[string]interface{}{"path": "x.txt"},
        &fakeEditTool{}, nil)
    require.Equal(t, 1, spy.calls)
    require.Equal(t, "fs_write", spy.lastCtx.ToolName)
    require.Equal(t, []string{"x.txt"}, spy.lastCtx.MutatedPaths)
}

func TestDerivePaths_TableDriven(t *testing.T) {
    cases := []struct{
        name   string
        tool   string
        params map[string]interface{}
        want   []string
    }{
        {"fs_write_single_path", "fs_write",
        map[string]interface{}{"path": "a.go"},
        []string{"a.go"}},
        {"fs_edit_single_path", "fs_edit", map[string]interface{}
        {"path": "a.go"}, []string{"a.go"}},
        {"smart_edit_single_path", "smart_edit",
        map[string]interface{}{"path": "a.go"},
        []string{"a.go"}},
        {"multiedit_dedup", "multiedit_commit",
        map[string]interface{}{
            "edits": []interface{}{
                map[string]interface{}{"path": "a.go"},
```

```
                map[string]interface{}{"path": "b.go"},
                map[string]interface{}{"path": "a.go"},  // dup
            }}, []string{"a.go", "b.go"}},
        {"mapping_edit_target_file", "mapping_edit",
        map[string]interface{}{"target_file": "x.go"},
        []string{"x.go"}},
        {"unknown_tool_fallthrough", "weird_tool",
        map[string]interface{}{}, nil},
    }
    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            got := derivePaths(tc.tool, tc.params)
            require.Equal(t, tc.want, got)
        })
    }
}
```

Non-obvious call: the path-derivation table is per-tool, NOT generic introspection. A generic "find all `path`-shaped strings" approach would over-trigger on unrelated args. Future tools that mutate files with novel param shapes need an explicit entry here; the fallthrough returns `nil` (the committer's porcelain-based discovery still works for those cases — it just commits everything dirty under the working dir).

Subject: `feat(P2-F22-T06): registry post-Execute fireAutoCommit hook + per-tool mutated-paths derivation (TDD).`

Task 7: /git_auto_commit slash command (TDD)

Files: new `helix_code/internal/commands/git_auto_commit_command.go`, new `helix_code/internal/commands/git_auto_commit_command_test.go`.

`git_auto_commit_command.go`:

```
type GitAutoCommitCommand struct {
    committer *autocommit.AutoCommitter
}

func NewGitAutoCommitCommand(c *autocommit.AutoCommitter)
        *GitAutoCommitCommand {
    return &GitAutoCommitCommand{committer: c}
}

func (c *GitAutoCommitCommand) Name() string         { return
        "git_auto_commit" }
func (c *GitAutoCommitCommand) Aliases() []string    { return
        nil }
func (c *GitAutoCommitCommand) Description() string {
    return "Show or change git auto-commit (per-edit) state."
}
```

```
func (c *GitAutoCommitCommand) Usage() string { return "/
        git_auto_commit [status|on|off|show]" }

func (c *GitAutoCommitCommand) Execute(ctx context.Context, cc
        *CommandContext) (*CommandResult, error) {
    sub := "status"
    if len(cc.Args) > 0 { sub = cc.Args[0] }
    switch sub {
    case "status":
        state := "off"
        if c.committer.Enabled() { state = "on" }
        repoState := "no"
        if c.committer.IsGitRepo() { repoState = "yes" }
        return &CommandResult{Output: fmt.Sprintf(
            "git_auto_commit: %s\ngit_repo: %s\nco-author
        trailer: %s",
            state, repoState, autocommit.CoAuthorTrailer)}, nil
    case "on":
        c.committer.SetEnabled(true)
        return &CommandResult{Output: "git_auto_commit → on"},
        nil
    case "off":
        c.committer.SetEnabled(false)
        return &CommandResult{Output: "git_auto_commit → off"},
        nil
    case "show":
        return &CommandResult{Output: fmt.Sprintf(
            "subject: <LLM-summarised, ≤72 chars>\n\n%s\n",
            autocommit.CoAuthorTrailer)}, nil
    default:
        return nil, fmt.Errorf("/git_auto_commit: unknown
        subcommand %q (want status|on|off|show)", sub)
    }
}
```

Failing tests FIRST:

```
func TestGitAutoCommit_Name_IsGitAutoCommit(t *testing.T) {
    require.Equal(t, "git_auto_commit",
        NewGitAutoCommitCommand(nil).Name())
}

func TestGitAutoCommit_Status_PrintsState(t *testing.T) {
    c := newTestCommitter(t, true)
    cmd := NewGitAutoCommitCommand(c)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "git_auto_commit: on")
```

```
}

func TestGitAutoCommit_Off_FlipsState(t *testing.T) {
    c := newTestCommitter(t, true)
    cmd := NewGitAutoCommitCommand(c)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"off"}})
    require.NoError(t, err)
    require.False(t, c.Enabled())
}

func TestGitAutoCommit_On_FlipsState(t *testing.T) {
    c := newTestCommitter(t, false)
    cmd := NewGitAutoCommitCommand(c)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"on"}})
    require.NoError(t, err)
    require.True(t, c.Enabled())
}

func TestGitAutoCommit_Show_PrintsTrailer(t *testing.T) {
    c := newTestCommitter(t, true)
    cmd := NewGitAutoCommitCommand(c)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"show"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "Co-Authored-By: HelixCode
        <noreply@helixcode.dev>")
}

func TestGitAutoCommit_UnknownSubcommand_Err(t *testing.T) {
    c := newTestCommitter(t, true)
    cmd := NewGitAutoCommitCommand(c)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"nope"}})
    require.Error(t, err)
}
```

Subject: `feat(P2-F22-T07): /git_auto_commit slash command (status/on/off/show) (TDD).`

Task 8: main.go wiring + integration test (TDD)

Files: modify `helix_code/cmd/cli/main.go`; new `helix_code/tests/integration/autocommit_test.go` (`//go:build integration`).

`main.go` changes (additive only):

```
// Resolve initial enabled state from env (default-on).
acEnabled := os.Getenv(autocommit.EnvVarName) != "off"

cwd, _ := os.Getwd()
c.autoCommitter = autocommit.NewAutoCommitter(autocommit.Options{
    Enabled:    acEnabled,
    Provider:   c.llmProvider,
    WorkingDir: cwd,
    Logger:     c.logger,
})
c.toolRegistry.SetAutoCommitter(c.autoCommitter)

if regErr :=
        c.commandRegistry.Register(commands.NewGitAutoCommitCommand(c.auto
        regErr != nil {
    log.Printf("git_auto_commit: register slash command failed:
        %v", regErr)
}
```

Failing integration tests FIRST (real registry + real F21 ApprovalManager + real WriteFileTool + real git tempdir):

```
//go:build integration

func TestAutoCommit_Integration_DefaultOn_RealEdit_RealCommit(t
        *testing.T) {
    dir := setupRealGitRepoWithInitialCommit(t)
    initialSHA := headSHA(t, dir)
    reg, _ := buildIntegrationRegistryAt(t, dir,
        approval.ModeAutoEdit, /*ac enabled=*/true)
    target := filepath.Join(dir, "x.txt")
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{"path": target, "content":
        "hello"})
    require.NoError(t, err)
    newSHA := headSHA(t, dir)
    require.NotEqual(t, initialSHA, newSHA)
    body := commitBody(t, dir)
    require.Contains(t, body, "Co-Authored-By: HelixCode
        <noreply@helixcode.dev>")
    require.Empty(t, strings.TrimSpace(porcelain(t, dir)))
}

func TestAutoCommit_Integration_EnvOff_NoCommit(t *testing.T) {
    dir := setupRealGitRepoWithInitialCommit(t)
    initialSHA := headSHA(t, dir)
    reg, _ := buildIntegrationRegistryAt(t, dir,
        approval.ModeAutoEdit, /*ac enabled=*/false)
    target := filepath.Join(dir, "x.txt")
```

```
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{"path": target, "content":
        "hello"})
    require.NoError(t, err)
    require.Equal(t, initialSHA, headSHA(t, dir))
    require.Contains(t, porcelain(t, dir), "x.txt")
}

func TestAutoCommit_Integration_RuntimeToggle(t *testing.T) {
    dir := setupRealGitRepoWithInitialCommit(t)
    initialSHA := headSHA(t, dir)
    reg, c := buildIntegrationRegistryAt(t, dir,
        approval.ModeAutoEdit, /*ac enabled=*/false)

    // First call (off) should NOT commit.
    target1 := filepath.Join(dir, "a.txt")
    _, _ = reg.Execute(context.Background(), "write_file",
        map[string]interface{}{"path": target1, "content": "a"})
    require.Equal(t, initialSHA, headSHA(t, dir))

    // Stage + manually commit so a.txt doesn't pollute the
        second call's diff.
    runGit(t, dir, "add", "a.txt")
    runGit(t, dir, "commit", "-m", "manual")
    midSHA := headSHA(t, dir)

    // Toggle on; next call SHOULD commit.
    c.SetEnabled(true)
    target2 := filepath.Join(dir, "b.txt")
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{"path": target2, "content": "b"})
    require.NoError(t, err)
    require.NotEqual(t, midSHA, headSHA(t, dir))
}

func TestAutoCommit_Integration_PerEditSkip_HonouredViaParam(t
        *testing.T) {
    dir := setupRealGitRepoWithInitialCommit(t)
    initialSHA := headSHA(t, dir)
    reg, _ := buildIntegrationRegistryAt(t, dir,
        approval.ModeAutoEdit, /*ac enabled=*/true)
    target := filepath.Join(dir, "x.txt")
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{
        "path": target, "content": "hello",
        autocommit.SkipParamKey: true,
    })
    require.NoError(t, err)
```

```
    require.Equal(t, initialSHA, headSHA(t, dir))
    require.Contains(t, porcelain(t, dir), "x.txt")
}

func TestAutoCommit_Integration_NotAGitRepo_NoOp(t *testing.T) {
    dir := t.TempDir()  // NOT a git repo
    reg, _ := buildIntegrationRegistryAt(t, dir,
        approval.ModeAutoEdit, /*ac enabled=*/true)
    target := filepath.Join(dir, "x.txt")
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{"path": target, "content":
        "hello"})
    require.NoError(t, err)
    // Tool succeeded; auto-commit silently skipped because not a
        git repo.
}

func TestAutoCommit_Integration_F21Denied_NoCommit(t *testing.T)
        {
    dir := setupRealGitRepoWithInitialCommit(t)
    initialSHA := headSHA(t, dir)
    reg, _ := buildIntegrationRegistryAt(t, dir,
        approval.ModeSuggest, /*ac enabled=*/true)
    target := filepath.Join(dir, "x.txt")
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{"path": target, "content":
        "hello"})
    require.Error(t, err)
    require.ErrorIs(t, err, approval.ErrApprovalRequired)
    // F21 denial → execErr != nil → fireAutoCommit not invoked.
    require.Equal(t, initialSHA, headSHA(t, dir))
    _, statErr := os.Stat(target)
    require.True(t, os.IsNotExist(statErr))
}
```

Subject: `feat(P2-F22-T08): main.go wiring (env + autocommit construct + registry hook + /git_auto_commit) + integration test.`

Task 9: Challenge harness + close-out + push 4 remotes non-force

Files: new `helix_code/tests/integration/cmd/p2f22_challenge/main.go`, new `challenges/p2-f22-aider-git-auto-commit/CHALLENGE.md`, new `challenges/p2-f22-aider-git-auto-commit/run.sh`.

Harness phases (per spec §6.3):

1. **PHASE-A: DEFAULT-ON-COMMITS-EDIT (always runs)** — env unset → committer enabled → real `WriteFileTool` through real registry against real git tempdir; assert (i) tool succeeds, (ii) `git log -1 --format=%H` changed, (iii) `git log -1 --format=%s` non-empty AND length ≤ 72, (iv) `git log -1 --format=%B` contains `Co-Authored-By: HelixCode <noreply@helixcode.dev>`, (v) `git status --porcelain` empty, (vi) `git show --stat HEAD` lists the written path.
2. **PHASE-B: LLM-SUMMARY-ACCURATE (always runs; uses fake llm.Provider returning sentinel `"FAKE_LLM_RESPONSE_42"`)** — assert commit subject equals the sentinel (proves LLM-call-then-use, not hardcoded fallback).
3. **PHASE-C: NON-EDIT-NO-OP (always runs)** — invoke `read_file` (LevelReadOnly); assert `git log -1 --format=%H` unchanged after the call.
4. **PHASE-D: ENV-OFF-NO-COMMIT (always runs)** — set `HELIXCODE_GIT_AUTO_COMMIT=off` before construct; invoke `WriteFileTool`; assert (i) tool succeeds, (ii) `git log -1 --format=%H` unchanged, (iii) `git status --porcelain` shows the file as dirty.
5. **PHASE-E: RUNTIME-TOGGLE (always runs)** — start off → `committer.SetEnabled(true)` → next call commits. Assert SHA before/after differential AND `committer.Enabled()` returns true after.
6. **PHASE-F: PER-EDIT-SKIP (always runs)** — invoke `WriteFileTool` with `params[autocommit.SkipParamKey] = true`; assert (i) tool succeeds, (ii) `git log -1 --format=%H` unchanged, (iii) `git status --porcelain` shows the file as dirty.

Optional **PHASE-G: SECRET-FILTER (runs)** — synthesise a diff containing fake `sk-` key; assert `git log -1 --format=%B` contains `[REDACTED]` and NOT the fake key.

Output skeleton ends with:

```
SUMMARY: PHASE-A=6/6 PASS; PHASE-B=3/3 PASS; PHASE-C=2/2 PASS; PHASE-D=3/3
         PHASE-E=4/4 PASS; PHASE-F=3/3 PASS; PHASE-G=2/2 PASS
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Anti-bluff smoke clean check appended. Verbatim output captured into `07_phase_2_evidence.md`. Dual commit (Challenges submodule + meta-repo bump).

`challenges/p2-f22-aider-git-auto-commit/run.sh` mirrors F19/F20/F21 structure: `cd HelixCode && go run ./tests/integration/cmd/p2f22_challenge/main.go`.

Close-out — tick all 9 items in PROGRESS, advance PROGRESS focus from F22 to "Phase 2 of CLI-Agent Fusion programme: F22 closed; F23 next candidate". Run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/autocommit internal/commands/
        git_auto_commit_command.go && echo BLUFF || echo clean
go test -count=1 ./internal/autocommit/...
go test -count=1 ./internal/commands/...
go test -count=1 ./internal/tools/...
go test -count=1 -tags=integration ./tests/integration/...
go mod tidy
```

```
git diff --exit-code go.mod  # MUST be no-op (zero new deps)
git diff --exit-code go.sum  # MUST be no-op
```

Cross-compile check:

```
cd HelixCode && GOOS=linux GOARCH=amd64 go build -o /tmp/
        helixcode-linux-amd64 ./cmd/server
ls -la /tmp/helixcode-linux-amd64
```

Commit `chore(P2-F22-T09): close out feature 22 — Aider Git Auto-Commit Per Change`. Push 4 remotes non-force (`origin`, `helixdev`, `vasic-digital`, `gitlab` per programme conventions). Request explicit user authorization at this step (CONST-043).

PROGRESS.md milestone entry (verbatim):

```
- 2026-05-06 — Feature 22 (Aider Git Auto-Commit Per Change) closed. 9 tas
  Real, end-to-end per-edit git auto-commit modelled on aider: one commit
  accepted edit (LevelEdit/LevelAll tools); LLM-summarised subject (50-72
  imperative voice) with deterministic fallback on LLM unavailability;
  Co-Authored-By: HelixCode <noreply@helixcode.dev> trailer appended to ev
  auto-commit. Default-on; opt-out via HELIXCODE_GIT_AUTO_COMMIT=off env,
  /git_auto_commit off slash, or per-edit _helix_skip_git_commit:true para
  Composes with F21 approval (only commits AFTER approval granted) and F04
  worktree (auto-commits work in subagent worktrees). Best-effort secret
  filter (CONST-042). Zero new external deps. Never pushes (CONST-043).
  [6+1 phase Challenge evidence summary].
```

Subject: `chore(P2-F22-T09): close out feature 22 — Aider Git Auto-Commit Per Change`.

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + constants (§3.3 + §10), T03 git wrapper (§3.3 `Git`), T04 summariser + secret filter (§3.4 prompt + §3.5 fallback + §5.2 #5), T05 committer pipeline (§4 data flow), T06 registry hook + path derivation (§3.5 derivation table), T07 /git_auto_commit slash (§4 + §3.2), T08 main.go wiring + integration test (§4 startup + §6.2), T09 Challenge 6+1 phases (§6.3 + §5.2 anti-bluff hot zone) + close-out.
2. **TDD:** every code task starts with failing tests. Types test pins constants byte-for-byte. Git wrapper tests use real `git init` + real commits (NO mocks of git). Summariser tests use an in-package fake `llm.Provider` returning sentinel responses. Committer tests use real tempdir + real git + real commits with positive evidence (`git log` SHA equality, `git status --porcelain` empty, co-author trailer presence). Registry test uses a spy committer to assert the hook fires only for LevelEdit/LevelAll AND honours `_helix_skip_git_commit:true`. Slash command tests use a constructed committer (no mocks). Integration tests wire the production registry + ApprovalManager + AutoCommitter end-to-end through real WriteFileTool.
3. **Type consistency:** `CommitContext`, `CommitResult`, `Options`, `AutoCommitter`, `MessageSummariser`, `LLMSummariser`, `DeterministicFallback`, `SecretFilter`, `Git`, `GitAutoCommitCommand`, sentinel errors (`ErrNotGitRepo`, `ErrCommitFailed`, `ErrLLMUnavailable`), constants

(`EnvVarName`, `CoAuthorTrailer`, `SkipParamKey`), command name `git_auto_commit`, env var `HELIXCODE_GIT_AUTO_COMMIT` — all match across spec §3 and plan T02-T08.

4. **Zero new external deps:** stdlib + existing testify/zap + in-tree `internal/llm.Provider`. `go mod tidy` after T08 produces no diff in `go.mod` or `go.sum`. T09's verification step asserts `git diff --exit-code go.{mod,sum}` is no-op.
5. **Anti-bluff (§5.2):** Challenge has SIX phases plus optional PHASE-G (always runs in time budget). Every phase records positive evidence: SHA equality (`git log -1 --format=%H` before/after differential), filesystem-state assertions (`git status --porcelain` empty/non-empty), trailer-presence assertion (`git log -1 --format=%B` contains `Co-Authored-By: HelixCode <noreply@helixcode.dev>`), `git show --stat HEAD` lists exactly the expected paths, LLM-fake-provider sentinel equality (PHASE-B), runtime-toggle observable in next-call SHA (PHASE-E). Byte-evidence mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.
6. **CONST-042:** secret filter runs over the LLM-generated subject before commit; unit tests assert each of four common patterns (AKIA, sk-, xoxb, gh[pousr]_) is replaced with `[REDACTED]`. The committer's logger NEVER logs the diff body at INFO level — only paths, SHA, and length. A unit test scans `internal/autocommit/*.go` for `logger\.Info\(.*\b(diff|body|content)\b` matches and FAILs on any hit.
7. **CONST-043:** F22 NEVER calls `git push` and the `Git` wrapper has no `Push` method. T09's close-out push requires explicit user authorization per CONST-043; auto-commits are commit-only.
8. **CONST-033:** F22 emits no shell commands beyond `git ...` invocations through `exec.CommandContext`. No suspend/reboot/halt commands.
9. **F21 integration:** `fireAutoCommit` runs ONLY when `execErr == nil`. F21's denial path returns `ErrApprovalRequired` from `Execute`, so `execErr != nil` and the auto-commit hook is structurally bypassed. Integration test `TestAutoCommit_Integration_F21Denied_NoCommit` asserts both directions (F21 denies → tool fails → no commit + no file).
10. **F04 worktree integration:** `WorkingDir` option is set per-CLI-instance to `os.Getwd()`; in a subagent worktree, the cwd IS the worktree path, so all `git` invocations operate on the worktree's git directory (which is a real git work tree pointed at the parent's `.git` via `gitdir:`). No special handling needed.
11. **Non-obvious call: per-tool path derivation table** (recorded in spec §11 #2 + plan T06). Generic introspection over `params` would over-trigger; the explicit table ensures only known mutating tools contribute paths. Unknown tools fall through to `nil` and the porcelain-based discovery path in `committer.MaybeCommit` covers them safely.
12. **Non-obvious call: post-Execute hook fires AFTER F13 LSP auto-trigger** (recorded in spec §11 #1). LSP doesn't depend on commit state; running it first lets diagnostics settle before the working tree gets committed.
13. **Non-obvious call:** `atomic.Bool` **for enabled flag** (recorded in spec §11 #4). Lock-free, single writer + many readers. Anti-bluff #4 (env honoured but slash ignored) is structurally impossible because every `MaybeCommit` begins with `if !c.enabled.Load() { return Skipped }`.
14. **Non-obvious call: default-on with literal** `off` **opt-out** (recorded in spec §11 #5). Typos default to safe-on. Slash `off` is the canonical runtime-off path.
15. **Non-obvious call: 5-second LLM timeout** (recorded in spec §11 #6). Bounded blocking; longer would let a stuck provider stall every commit.
16. **Non-obvious call: subject length cap at 72 chars** (recorded in spec §11 #7). Git's de-facto subject convention. Truncation, no ellipsis.
17. **Non-obvious call: co-author trailer is appended to EVERY auto-commit unconditionally** (recorded in spec §11 #8). Including fallback-message commits. Q3=A is unambiguous. Tests pin this byte-for-byte.

18. **Non-obvious call: auto-commit failure swallowed at registry boundary** (recorded in spec §11 #9). NEVER propagated as a tool error. Logged at WARN.
19. **Non-obvious call:** `RequiresApproval()` **filter EXCLUDES LevelReadOnly AND LevelRun** (recorded in spec §11 #10). Read tools don't touch files; shell tools (LevelRun) may touch files but the user is supervising the shell already, and shell-induced commits would mix with the user's own workflow.
20. **Non-obvious call: F21 + F22 compose monotonically** (recorded in spec §11 #11). F22's hook runs only when F21 allowed and the tool succeeded. No divergent paths.
21. **Non-obvious call:** `git diff --staged` **(not** `--unstaged`**)** (recorded in spec §11 #14). By the time `summariser.Summarise` runs, paths are already staged. Using `--staged` ensures the LLM sees exactly what's about to be committed.
22. **Non-obvious call: shell-out** `git`**, NOT go-git** (recorded in spec §11 #15). Honours user's `~/.gitconfig`, hooks, signing, aliases. The wrapper is intentionally thin.
23. **Second Phase 2 feature:** F22 is the second Phase 2 feature after F21. T01 advances PROGRESS.md from "F21 closed" to "F22 in flight"; appends F22 evidence header to existing `07_phase_2_evidence.md` (created in F21-T01).